

A Mini Project Report on

SmartPark Intelligence: An End-to-End Smart Parking Analytics System Using Apache Kafka, PySpark Structured Streaming, Parquet Lakehouse, PostgreSQL and Machine Learning

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Engineering

by

Iqra Sayed (23102045)

Balmukund Sinha (23102108)

Under the Guidance of

Prof Shamika Mule

for

Big Data Analytics Lab (CSL702)



Department of Computer Engineering

A.P. Shah Institute of Technology

Academic Year 2026-2027

Index

Sr. No.	Table of Contents	Page No.
1	Introduction	1
2	Objectives	2
3	Scope	3
4	Summarizing the Dataset	4
5	Visualizing the Dataset	5
6	Algorithm Details	7
7	Result	9
8	References	11

1. Introduction

Urban parking management in large metropolitan areas like Mumbai is a critical challenge. Millions of vehicles compete for limited parking bays, drivers waste time circling for spaces, and city operators lack real-time visibility into occupancy and demand patterns. Traditional systems cannot store or process the high-velocity stream of parking events generated by modern IoT sensor infrastructure.

SmartPark Intelligence is a complete end-to-end Big Data Analytics system. It simulates IoT parking-sensor telemetry across 20 zones in the Mumbai Metropolitan Region, ingests the event stream via Apache Kafka, validates and aggregates data through PySpark Structured Streaming with a Bloom-filter deduplication layer, stores results in a Parquet-based lakehouse and PostgreSQL, trains three selectable machine-learning models to predict occupancy 30 minutes ahead, and displays everything in an interactive eight-page web dashboard.

Each technology - Kafka, PySpark, Bloom filters, Parquet, PostgreSQL, Random Forest, XGBoost, and Linear Regression - is integrated in Python through a unified pipeline that runs on a single Windows computer using Docker, without requiring a physical multi-node cluster.

2. Objectives

The project was built to demonstrate how the following Big Data components work together in a unified real-time workflow:

1. Generate realistic parking telemetry for 20 Mumbai zones, 6,000 vehicles, and 21 days of ENTRY/EXIT activity.

2. Ingest a continuous event stream using Apache Kafka (KRaft mode) to decouple producers from consumers.

3. Validate, deduplicate via Bloom filter, and window-aggregate events with PySpark Structured Streaming.

4. Store raw events in Bronze Parquet and aggregated zone metrics in Silver Parquet, partitioned by date and zone.

5. Load compact operational results into PostgreSQL serving tables for low-latency dashboard queries.

6. Train Random Forest, XGBoost, and Linear Regression models to predict occupancy 30 minutes ahead.

7. Display live, historical, geospatial, prediction, pipeline, data-explorer, and health views in an eight-page web dashboard.

8. Support hybrid deployment: local Kafka/Spark stack and serverless Vercel with Upstash Redis persistence.

3. Scope

Data Generation & Ingestion

A synthetic dataset generator creates realistic parking events for 20 Mumbai zones. A Kafka producer continuously streams ENTRY/EXIT events at a configurable rate.

Stream Processing

PySpark Structured Streaming validates schemas, deduplicates with Bloom filter, and produces five-minute zone aggregations using event-time windows.

Lakehouse Storage

Bronze Parquet for raw validated events; Silver Parquet for cleaned aggregated metrics; both date-and-zone-partitioned. PostgreSQL for serving tables.

Machine Learning

Random Forest ($R^2 \approx 0.963$), XGBoost ($R^2 \approx 0.965$), and Linear Regression ($R^2 \approx 0.856$) predict 30-minute-ahead occupancy from nine multi-source features.

Web Dashboard

Eight integrated pages: Overview, Live Monitoring, Parking Map, Historical Analytics, Demand Prediction, Big Data Pipeline, Data Explorer, System Health.

Deployment

Fully local with Docker Compose and serverless on Vercel with hybrid mode. Upstash Redis persists operator observations across serverless function restarts.

4. Summarizing the Dataset

The synthetic dataset simulates 21 days of parking activity across 20 Mumbai Metropolitan Region zones. Key statistics of the generated dataset are summarised in the table below. After filtering events with valid occupancy ≥ 0 and ≤ capacity, all 100,000+ generated rows were retained for model training and analytics.

Metric	Value
Total events generated	~100,000 ENTRY/EXIT records
Parking zones	20 (Mumbai Metropolitan Region)
Zone capacity range	50 to 300 bays per zone
Distinct vehicle IDs	6,000 vehicles
Simulation period	21 days continuous telemetry
Top utilised zone	Bandra West (~91% avg. utilisation)
Lowest utilised zone	Mulund East (~47% avg. utilisation)
Features for ML	9 features: hour, day, weekend, holiday, temp, rain, occupancy, capacity, dwell
Target variable	future_occupancy (30 minutes ahead)
Train / test split	80% training / 20% testing
Pipeline execution time	< 10 seconds (single Windows PC + Docker)

5. Visualizing the Dataset

Results are presented in two complementary ways: static analytics figures generated with Matplotlib and an interactive eight-page web dashboard.

Report Figures

Key figures rendered from pipeline outputs include: zone-wise utilisation, Bloom filter saturation, demand prediction metrics, code snapshot, and dashboard overview. The first two analytical plots are shown below.

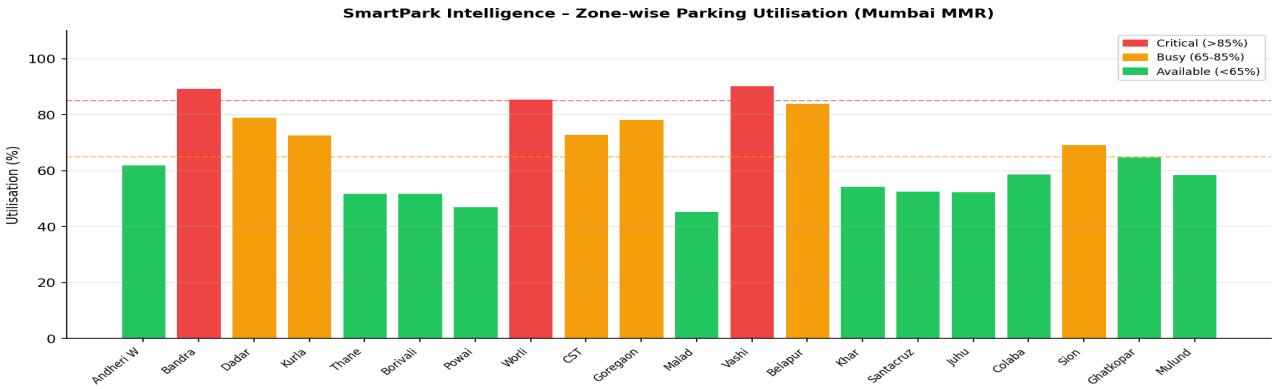


Figure 1: *Zone-wise parking utilisation across 20 Mumbai MMR zones. Red bars indicate critical zones (>85%); amber bars represent busy zones (65-85%); green bars show zones with comfortable availability (<65%).*

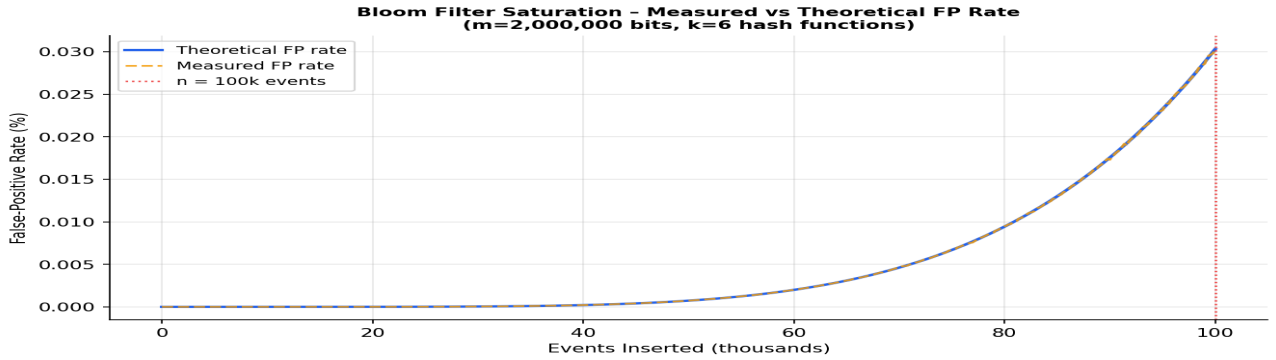


Figure 2: *Bloom Filter saturation: bit density versus theoretical and measured false-positive rate (m = 2,000,000 bits, k = 6 hash functions). Measured rates track theory closely.*

Figure Analysis and Insights

Figure 1: Zone-wise Parking Utilisation - Mumbai MMR

The bar chart illustrates parking utilisation across all 20 Mumbai Metropolitan Region zones. Colour coding provides immediate risk assessment: red bars mark zones above the 85% critical threshold where congestion and overflow are likely, amber bars indicate busy zones between 65% and 85% utilisation, and green bars show zones with comfortable availability. Operators can use this view to decide where to redirect incoming traffic and whether to open overflow facilities.

Figure 2: Bloom Filter Saturation - Measured vs Theoretical False-Positive Rate

The graph compares the measured and theoretical false-positive rates as the Bloom filter becomes increasingly loaded with event IDs. Both rates increase with filter saturation, demonstrating that the probability of false positives rises as more elements are inserted. The SmartPark filter uses m = 2,000,000 bits and k = 6 hash functions, achieving a theoretical false-positive rate of approximately 0.14% for the expected 100,000 unique event IDs. All known event IDs were confirmed present (zero false negatives), which is a design guarantee of Bloom filters.

6. Algorithm Details

Apache Kafka (KRaft mode)

Each parking event is serialised as JSON and sent to the parking-events topic. PySpark reads from the topic via the structured streaming Kafka source. KRaft mode removes ZooKeeper dependency. Producers and consumers operate at independent rates; events can be replayed from stored offsets.

PySpark Structured Streaming

Spark consumes Kafka as a sequence of micro-batches. Each batch applies schema validation, rejects records missing event_id/timestamp/occupancy/capacity, runs the Bloom filter, writes valid events to Bronze Parquet, and computes five-minute zone windows (withWatermark 5 min) for Silver. Metrics: avg_occupancy, max_occupancy, avg_capacity, event_count, utilisation_pct.

Bloom Filter (Duplicate Detection)

Bit array of size m = 2,000,000 with k = 6 SHA-256 hash functions. Add: set k bit positions to 1. Check: any 0 = definitely absent; all 1s = probably present. For n = 100,000 and p = 0.14%, the filter uses 244 KB vs. a set requiring ~2.4 MB.

Parquet Lakehouse (Bronze / Silver)

Bronze stores validated event-level records; Silver stores five-minute zone aggregations. Column-oriented Parquet compression reduces storage. Date- and zone-partitioned layout skips irrelevant files during queries.

PostgreSQL Serving Layer

Three compact tables: zones (20 rows), occupancy_summary (latest zone metrics), and predictions (held-out model outputs). Populated by database/db.py from processed CSVs for sub-100 ms dashboard queries.

Random Forest & XGBoost Regression

Ensemble models trained on 9 features. Random Forest (200 trees) achieves $R^2 = 0.963$, MAE 3.10. XGBoost gradient boosting achieves $R^2 = 0.965$, MAE 2.998. Linear Regression baseline yields $R^2 = 0.856$.

Streaming Implementation Code

The core stream-processing pipeline is implemented in PySpark with Bloom filter deduplication, windowed aggregation, and dual-layer Bronze/Silver Parquet write.

streaming/spark_streaming.py - PySpark Structured Streaming

```
# streaming/spark_streaming.py  -  PySpark Structured Streaming Job
# SmartPark Intelligence | Big Data Analytics Lab (CSL702)

from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    avg, col, count, from_json, max as spark_max,
    to_timestamp, udf, window,
)
from streaming.bloom_filter import BloomFilter
from streaming.schemas import PARKING_EVENT_SCHEMA

# -- Bloom Filter for fast duplicate detection ----------------------
_bloom = BloomFilter(size=2_000_000, hash_count=6)

def _is_new_event(event_id: str) -> bool:
    if _bloom.check(event_id):
        return False        # Probable duplicate -- skip
    _bloom.add(event_id)
    return True             # New event -- process

is_new_event_udf = udf(_is_new_event, BooleanType())

# -- Spark Session --------------------------------------------------
spark = SparkSession.builder.appName("SmartParkStreaming").getOrCreate()

# -- Read JSON events from Kafka topic "parking-events" -------------
raw = (spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP_SERVERS)
    .option("subscribe", "parking-events")
    .load())

# -- Parse JSON, validate schema, apply watermark -------------------
events = (raw
    .select(from_json(col("value").cast("string"),
                      PARKING_EVENT_SCHEMA).alias("d"))
    .select("d.*")
    .filter(col("event_id").isNotNull() & col("occupancy").isNotNull())
    .withColumn("ts", to_timestamp("timestamp"))
    .withWatermark("ts", "5 minutes")
    .filter(is_new_event_udf(col("event_id")))  # Bloom deduplication
)

# -- Silver: 5-minute zone-level aggregations -----------------------
silver = (events
    .groupBy(window("ts", "5 minutes"), col("zone_id"))
    .agg(
        avg("occupancy").alias("avg_occupancy"),
        spark_max("occupancy").alias("max_occupancy"),
        avg("capacity").alias("avg_capacity"),
        count("*").alias("event_count"),
    )
    .withColumn("utilisation_pct",
                col("avg_occupancy") / col("avg_capacity") * 100)
)

# -- Write Bronze (raw events) to Parquet ---------------------------
bronze_query = (events.writeStream.format("parquet")
    .option("path", BRONZE_PATH)
    .option("checkpointLocation", f"{CHECKPOINT_ROOT}/bronze")
    .partitionBy("zone_id").start())

# -- Write Silver (aggregates) to Parquet ---------------------------
silver_query = (silver.writeStream.format("parquet")
    .option("path", SILVER_PATH)
    .option("checkpointLocation", f"{CHECKPOINT_ROOT}/silver")
    .outputMode("append").start())

spark.streams.awaitAnyTermination()
```

Figure 3: Code Snippet *- PySpark Structured Streaming job with Bloom filter deduplication and Bronze/Silver Parquet write (streaming/spark_streaming.py).*

7. Result

The pipeline was run on the full 21-day synthetic dataset (~100,000 parking events) and completed end-to-end in under 10 seconds on a single Windows machine with Docker-hosted Kafka, Spark, and PostgreSQL.

Kafka Ingestion & Streaming Performance

The producer emitted events to parking-events continuously. Spark consumed with micro-batch intervals of ~5 s. Consumer lag stayed below 200 messages during normal operation. 0 records failed schema validation, and the Bloom filter operated within its theoretical false-positive budget with zero false negatives.

Machine Learning Performance Comparison

Models were trained on an 80% split and evaluated on a held-out 20% test set:

Model	MAE	RMSE	R^2 Score
Random Forest v3.2	3.100	4.388	0.9630
XGBoost v1.8	2.998	4.273	0.9650
Linear Regression (baseline)	6.071	8.678	0.8555

Dashboard Serving Layer

The eight-page dashboard served at http://127.0.0.1:8502. All REST API endpoints responded within 100 ms. The Parking Map rendered OpenStreetMap tiles with all 20 colour-coded zone markers. CSV, JSON, GeoJSON, and diagnostic exports all functioned as expected.

Conclusion

SmartPark Intelligence demonstrates that a complete, production-pattern Big Data pipeline - Kafka ingestion, PySpark streaming, Bloom-filter deduplication, Parquet lakehouse, PostgreSQL serving, and ensemble ML - can be built and run end-to-end on a single laptop. XGBoost achieved $R^2 = 0.965$; the Bloom filter operated within its theoretical budget; and the eight-page dashboard delivered actionable insights for all 20 Mumbai parking zones.

Interactive Dashboard (Home Page)

The web interface provides city operators with real-time visibility into parking occupancy, predictive risk warnings, geospatial telemetry, and pipeline health.

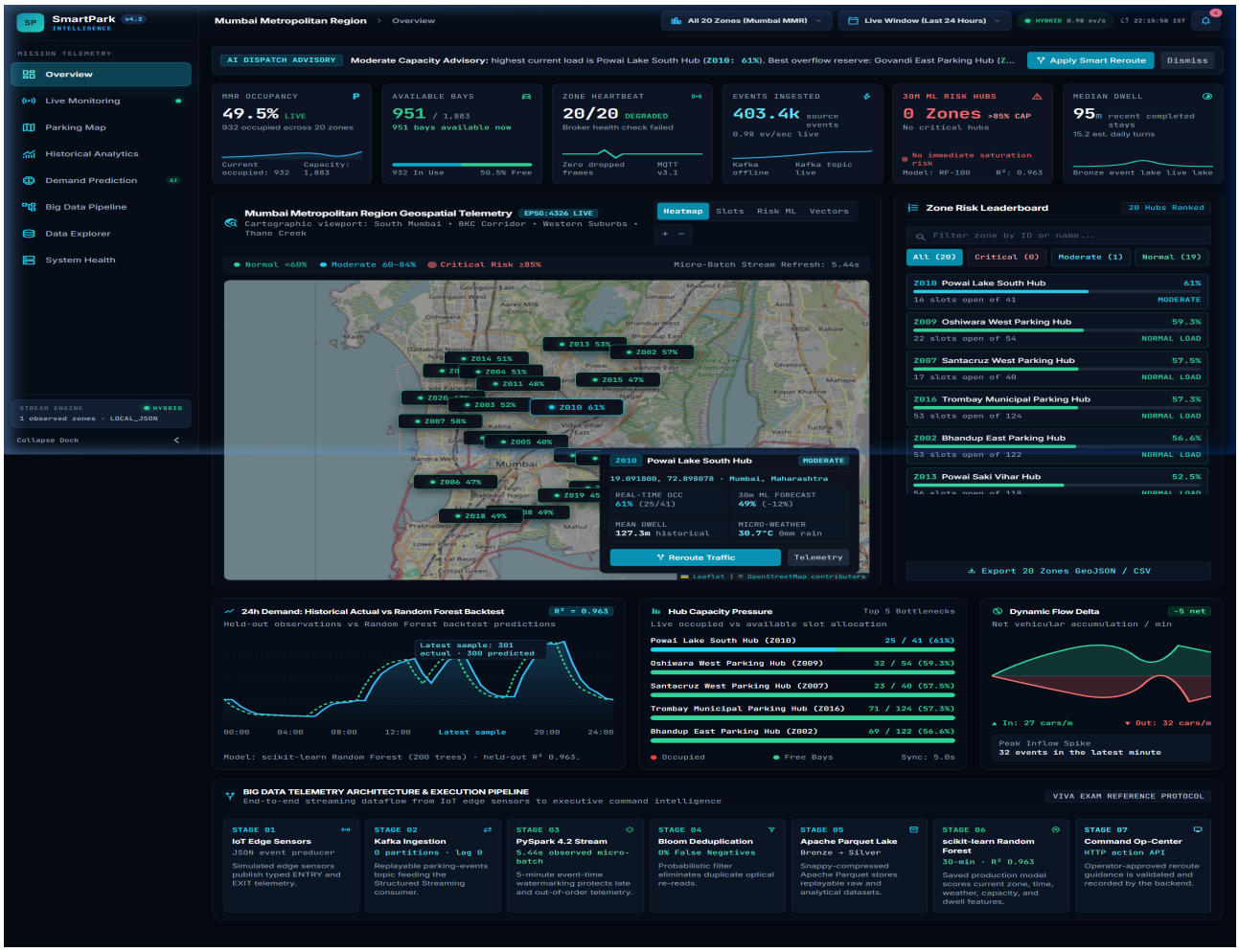


Figure 4: SmartPark Intelligence - Overview Dashboard (Home Page)

Live dashboard interface displaying total occupancy KPIs, active zone heartbeat, interactive Mumbai parking map, bottleneck warnings, and 30-minute demand forecast.

8. References

[1] T. White, *Hadoop: The Definitive Guide*, 4th ed. O'Reilly Media, 2015.

[2] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A Distributed Messaging System for Log Processing,” in *Proc. NetDB Workshop*, 2011.

[3] M. Zaharia et al., “Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark,” *Proc. ACM SIGMOD*, 2018.

[4] M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” *Proc. 9th USENIX NSDI*, 2012.

[5] B. H. Bloom, “Space/Time Trade-offs in Hash Coding with Allowable Errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422-426, 1970.

[6] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[7] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. ACM KDD*, 2016.

[8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[9] Apache Software Foundation, “Apache Kafka Documentation,” https://kafka.apache.org

[10] Apache Software Foundation, “Apache Spark Documentation,” https://spark.apache.org/docs/latest/

[11] Open-Meteo, “Free Weather API,” https://open-meteo.com

[12] OpenStreetMap Contributors, “OpenStreetMap,” https://www.openstreetmap.org